SYSTEM DESIGN DOCUMENT

TaskAnalyzer — Analisador de Métricas e Produtividade de Tarefas

Bootcamp III · Análise e Desenvolvimento de Sistemas · Versão 2.1 · 24/10/2026

1. Finalidade e escopo

Este documento descreve o desenho técnico do repositório sdd-task-analyzer com base na especificação SDD e nas evidências de execução compartilhadas. O TaskAnalyzer é um módulo Python que recebe tarefas estruturadas e calcula métricas agregadas de conclusão, atraso e distribuição por prioridade/status. As evidências disponíveis demonstram testes locais, execução containerizada e um workflow de GitHub Actions bem-sucedido; não demonstram implantação em produção nem auditoria completa de segurança.

2. Objetivos e limites funcionais

• Calcular tempo médio de conclusão geral e por prioridade.
• Calcular taxa de atraso geral e por prioridade.
• Contabilizar tarefas analisadas e indicadores por prioridade/status.
• Tratar inconsistências cronológicas e evitar divisão por zero.
• Preservar o contrato público e manter especificação, código e testes alinhados.

3. Componentes e responsabilidades

Artefato	Responsabilidade
specs/task_analyzer_spec.md	Contrato de negócio, entradas, saídas, regras e cenários de aceite.
specs/requirements.yaml	Requisitos funcionais/não funcionais e critérios de aceitação estruturados.
src/task_analyzer.py	Processamento de tarefas, validações e cálculo das métricas.
tests/test_harness.py	Testes automatizados de sucesso, inconsistência e casos de borda.
CONTEXT_RULES.md	Diretrizes arquiteturais, proibições e regras para uso/revisão de IA.
.github/workflows/ci.yml	Checkout, configuração de Python, instalação de dependências e execução de pytest.
Dockerfile	Imagem Python, diretório de trabalho, instalação de dependências, cópia dos arquivos e comando de teste.
docker-compose.yml	Configuração do serviço task-analyzer e construção a partir do Dockerfile.
README.md / requirements.txt	Instruções de uso e lista de dependências.

4. Fluxos de execução

Fluxo funcional: lista de tarefas → validação → seleção de tarefas concluídas com data de conclusão válida → cálculo de médias e taxas → agregação por prioridade/status → retorno estruturado.

Fluxo CI: push, pull request ou acionamento manual → GitHub Actions obtém o código → configura Python 3.11 → instala requirements.txt → executa pytest → publica o resultado do job.

Fluxo Docker: `docker compose build` constrói a imagem; `docker compose config` mostra a configuração resolvida; `docker compose up` inicia o contêiner e executa a suíte de testes conforme o comando padrão.

5. Contrato de dados

Entradas: cada tarefa contém `id`, `titulo`, `prioridade`, `data_criacao`, `data_conclusao`, `status` e `data_limite`. Os campos obrigatórios e opcionais, os tipos aceitos e os valores autorizados estão definidos em `specs/task_analyzer_spec.md`.

O campo `data_limite` é opcional e pode ser nulo. Quando informado, permite comparar a data de conclusão com o prazo estipulado e identificar tarefas concluídas com atraso.

Saídas: `tempo_medio_conclusao_geral`, `tempo_medio_por_prioridade`, `taxa_atraso_geral`, `taxa_atraso_por_prioridade`, `total_tarefas_analisadas` e `indicadores_por_prioridade`.

Consistência contratual: a especificação original prevê o campo `data_limite`. A implementação, os testes e a documentação devem permanecer alinhados quanto à interpretação desse campo e à regra usada para calcular atrasos.

6. Regras de negócio e aceite

• Médias e taxas consideram estritamente tarefas com status `Concluida` e `data_conclusao` válida.
• `data_conclusao` não pode ser anterior a `data_criacao`; a inconsistência deve ser tratada como exceção específica ou equivalente previsto no contrato.
• Lista vazia ou ausência de tarefas concluídas não deve provocar divisão por zero; métricas aplicáveis retornam valores seguros, como 0.0.
• `indicadores_por_prioridade` contabiliza tarefas por prioridade e status.
• Assinaturas de funções públicas não devem mudar sem revisão formal do contrato.
• Código novo deve vir acompanhado de testes automatizados; testes contratuais não devem ser burlados ou removidos.

7. Matriz de rastreabilidade dos testes

Teste evidenciado	Propósito	Resultado
test_cenario_1_sucesso_metricas_corretas	Validar métricas com dados válidos.	Aprovado
test_cenario_2_excecao_datas_inconsistentes	Validar tratamento de datas inconsistentes.	Aprovado
test_caso_de_borda_lista_vazia	Validar lista vazia.	Aprovado
test_caso_de_borda_apenas_pendentes	Validar ausência de tarefas concluídas.	Aprovado

Os quatro testes foram exibidos no terminal durante a execução local por Docker Compose, com código de saída 0. Isso comprova somente os cenários executados, não cobertura exaustiva.

8. Governança SDD e revisão humana

A especificação SDD é a referência para as decisões de implementação. Alterações devem ser pequenas, descritivas e rastreáveis por commits; pull requests e revisão humana devem ser usados para mudanças relevantes. Código sugerido por IA é uma proposta sujeita a leitura, comparação com o contrato e testes. A pessoa responsável deve compreender e conseguir explicar o que foi entregue.

As regras de contexto estabelecem Python 3.11+, type hints, responsabilidade única, PEP 8, docstrings no padrão Google, funções coesas, exceções específicas e logging. Também proíbem dependências não autorizadas, alterações aos testes contratuais, persistência em arquivos ou bancos nesta etapa, mudanças não aprovadas em assinaturas públicas e código novo sem testes correspondentes.

9. Segurança, privacidade e limites

Não incluir credenciais, tokens, segredos ou dados pessoais desnecessários no repositório público, prompts ou logs. A aprovação dos testes não substitui uma revisão de segurança. Não foi apresentada evidência de implantação em produção, análise estática, varredura de vulnerabilidades, auditoria automatizada de dependências ou assinatura da imagem; esses itens são melhorias futuras, não controles já comprovados.

As métricas devem ser interpretadas no contexto. Não constituem, isoladamente, uma avaliação completa da produtividade de uma pessoa, pois representam apenas os dados e regras definidos no contrato.

10. Evidências e limitações

As evidências compartilhadas mostram docker compose build concluído, docker compose config sem erro, docker compose up executando quatro testes aprovados com código 0 e uma execução do GitHub Actions com status de sucesso. Recomenda-se guardar as evidências e associá-las ao commit correspondente. A suíte atual é inicial e não comprova todos os comportamentos possíveis.

11. Melhorias priorizadas

Alta: confirmar e manter alinhada a regra de cálculo de atraso quando `data_limite` estiver ausente ou nula; conferir validação de campos obrigatórios; manter README e comandos coerentes com o que foi executado.
Média: ampliar testes para tipos inesperados, campos ausentes, fusos horários e combinações de status/prioridade; adicionar linting, análise estática e auditoria de dependências.
Futura: avaliar usuário não privilegiado no contêiner, varredura de vulnerabilidades e imagens fixadas por digest.

12. Conclusão

A arquitetura separa contrato, implementação, testes, governança e automação de execução. As evidências demonstram validação inicial com quatro testes, CI e execução containerizada. A documentação passa a reconhecer corretamente que a especificação inclui o campo opcional `data_limite`; ainda é necessário confirmar que sua regra de cálculo está alinhada entre código e testes e realizar revisão humana antes da entrega.

Documento técnico baseado na especificação SDD fornecida e nas evidências de execução compartilhadas. Revisar após qualquer mudança no contrato, código ou ambiente.